Lido Circuit Breaker Audit Report

Prepared by Cyfrin

Version 2.0

Lead Auditors

Dacian

SBSecurity (Blckhv, Slavcheww)

April 15, 2026

Contents

1 About Cyfrin **2**

2 Disclaimer **2**

3 Risk Classification **2**

4 Protocol Summary **2**

5 Audit Scope **3**

6 Executive Summary **3**

7 Findings **6**

7.1 Informational . . . 6

7.1.1 Missing named mapping value parameters in `Registry.Storage` . . . 6

7.1.2 Single pauser per pausable combined with heartbeat expiry lockout leaves zero emergency coverage during liveness gaps . . . 6

7.2 Gas Optimization . . . 8

7.2.1 Emit before state change to eliminate temporary variable . . . 8

7.2.2 Solidity optimizer is not enabled . . . 8

7.2.3 Redundant storage read of `registry.pauser[_pausable]` in `CircuitBreaker::pause` . . . 9

1 About Cyfrin

Cyfrin is a Web3 security company dedicated to bringing industry-leading protection and education to our partners and their projects. Our goal is to create a safe, reliable, and transparent environment for everyone in Web3 and DeFi. Learn more about us at cyfrin.io.

2 Disclaimer

The Cyfrin team makes every effort to find as many vulnerabilities in the code as possible in the given time but holds no responsibility for the findings in this document. A security audit by the team does not endorse the underlying business or product. The audit was time-boxed and the review of the code was solely on the security aspects of the in-scope code being audited.

3 Risk Classification

	Impact: High	**Impact: Medium**	**Impact: Low**
Likelihood: High	Critical	High	Medium
Likelihood: Medium	High	Medium	Low
Likelihood: Low	Medium	Low	Low

4 Protocol Summary

`CircuitBreaker` is an emergency pause manager for the Lido protocol — a single contract that lets designated pausers pause critical contracts on behalf of the admin.

There are two roles: the **admin** (Lido Agent) and **pausers** (multisig committees such as a standard Gnosis multisig). The admin assigns one pauser to each pausable contract. In an emergency, the pauser triggers a pause on a contract address for a globally configured pause duration. After a successful pause the pauser assignment is cleared — the admin must explicitly re-assign before the contract can be paused again through `CircuitBreaker`.

Pausers must periodically send a heartbeat to prove liveness. A pauser whose heartbeat has expired cannot pause or refresh their heartbeat; only the admin can restore an expired pauser by re-registering them.

The protocol consists of two contracts:

- `CircuitBreaker` — the main contract with an immutable admin, configurable pause duration and heartbeat interval (within deployment-time min/max bounds), and functions for admin configuration (`setPauseDuration`, `setHeartbeatInterval`, `registerPauser`), pauser liveness (`heartbeat`), and emergency pause (`pause`). Uses a transient storage reentrancy guard (EIP-1153) on the `pause` function to prevent a malicious pausable from re-entering to trigger additional pauses. Must be granted `PAUSER_ROLE` on the target pausable contracts in order to call `pauseFor`
- `Registry` — a library managing an enumerable pausable-to-pauser registry using a swap-and-pop pattern for O(1) additions and removals. Maintains bidirectional consistency between a `pausables` array, `oneBasedIndex` mapping, `pauser` mapping, and per-pauser `pausableCount`. Idempotent unregistration is supported by design to prevent a pauser from griefing the admin by frontrunning an unregistration transaction with a `pause` call

The pausable contracts this system is expected to manage on Ethereum mainnet include WithdrawalQueue, ValidatorExitBusOracle, TriggerableWithdrawalsGateway, VaultHub, PredepositGuarantee, and CSFeeOracle — all of which implement `pauseFor(uint256)` with `onlyRole(PAUSE_ROLE)` and `isPaused()` via Lido's `PausableUntil` base contract.

Paused contracts can be unpaused by `RESUME_ROLE` which is expected to be given to Lido Agent on-demand and then revoked immediately after. In case of `CircuitBreaker`, it doesn't need the role because the pause is temporary and auto-resumes after the pause duration has expired.

5 Audit Scope

The audit scope was limited to:

```
src/CircuitBreaker.sol
src/Registry.sol
```

6 Executive Summary

Over the course of 2 days, the Cyfrin team conducted an audit on the Circuit Breaker code provided by Lido. The findings consist of 2 Informational & 3 Gas Optimizations.

Centralization Risks

The immutable `ADMIN` (Lido Agent) has significant centralized control:

- Sole authority over pauser assignment - only `ADMIN` can register, replace, or unregister pausers. If the DAO agent is compromised or governance is captured, an attacker could unregister all pausers (disabling emergency pause capability) or register malicious pausers
- Controls pause parameters — `ADMIN` sets `pauseDuration` and `heartbeatInterval` within deployment-time bounds. Misconfiguration (e.g., very low heartbeat interval causing all pausers to expire, or very low pause duration making pauses ineffective) could degrade emergency response capability
- Only entity that can revive expired pausers — if a pauser's heartbeat expires, only `ADMIN` can restore coverage by re-registering them. Combined with the single-pauser-per-pausable design, this creates a window of zero emergency pause coverage during liveness gaps (see I-2)
- No admin transfer mechanism — `ADMIN` is immutable. If the Lido Agent address needs to change, `CircuitBreaker` must be redeployed and all pausable contracts must re-grant `PAUSE_ROLE` to the new deployment

Protocol Invariants

The following invariants were verified using the Certora Prover (38 of 41 rules verified; 3 known limitations marked with * below):

Registry Enumerable Set Integrity

1. For every valid index `i < pausables.length`: `oneBasedIndex[pausables[i]] == i + 1`
2. For every address `p` with `oneBasedIndex[p] != 0`: `pausables[oneBasedIndex[p] - 1] == p`
3. For address `p != address(0)`: `pauser[p] != address(0)` if and only if `oneBasedIndex[p] != 0` *
4. For every valid index `i < pausables.length`: `pauser[pausables[i]] != address(0)`
5. For all valid indices `i != j`: `pausables[i] != pausables[j]` (no duplicates)
6. For address `p` not in `pausables[]`: `oneBasedIndex[p] == 0` and `pauser[p] == address(0)` *
7. For any pauser `q`: `pausableCount[q]` equals the count of pausable addresses where `pauser[p] == q`
8. Sum of `pausableCount[q]` across all pausers equals `pausables.length` *
9. `pausableCount[address(0)] == 0`
10. `address(0)` is never in `pausables[]`

Access Control

11. `pauseDuration` changes only when `msg.sender == ADMIN`
12. `heartbeatInterval` changes only when `msg.sender == ADMIN`
13. Registry state changes only via `registerPauser` (ADMIN) or `pause` (authorized pauser)
14. `heartbeatExpiry[p]` changes only via `registerPauser` (ADMIN), `heartbeat` (self), or `pause` (self)

Bounds Enforcement

15. `MIN_PAUSE_DURATION <= pauseDuration <= MAX_PAUSE_DURATION` — always
16. `MIN_HEARTBEAT_INTERVAL <= heartbeatInterval <= MAX_HEARTBEAT_INTERVAL` — always

Heartbeat Liveness

17. `heartbeat()` reverts if `block.timestamp >= heartbeatExpiry[msg.sender]`
18. `pause()` reverts if `block.timestamp >= heartbeatExpiry[msg.sender]`
19. Only ADMIN can refresh an expired pauser's heartbeat (via `registerPauser`)
20. After heartbeat refresh: `heartbeatExpiry[pauser] == block.timestamp + heartbeatInterval`

Single-Use Pause

21. After successful `pause(_pausable)`: `pauser[_pausable] == address(0)` (consumed)
22. After successful `pause(_pausable)`: `pausableCount[msg.sender]` decreased by 1

Revert Conditions

23. `setPauseDuration`, `setHeartbeatInterval`, `registerPauser` revert if `msg.sender != ADMIN`
24. `heartbeat()` reverts if `pausableCount[msg.sender] == 0`
25. `pause(_pausable)` reverts if `msg.sender != pauser[_pausable]`
26. `registerPauser(_pausable, _)` reverts if `_pausable == address(0)`

Edge Cases

27. Idempotent unregistration: `registerPauser(p, 0)` when already unregistered is a net no-op
28. Replace-with-self: `registerPauser(p, q)` when `pauser[p] == q` preserves `pausableCount[q]`

Liveness

29. A registered, live pauser CAN successfully call `pause` (happy path reachable)
30. A registered, live pauser CAN successfully call `heartbeat`

* *Known limitation*: These 3 invariants are verified for all functions except `registerPauser`, where the Certora Prover cannot close the inductive proof due to circular dependencies between the enumerable set structural invariants. The properties are correct by manual analysis and verified for all other code paths including `pause` (which also modifies registry state). See `certora/README.md` for details.

Post-Audit Change Review

After the audit had concluded we were asked to review commit b4b2fbc which cleared the heartbeat when a pauser was left with no registered contracts, causing `CircuitBreaker::isPauserLive` to return `false` in this case. There were no issues found with this commit.

Summary

Project Name	Circuit Breaker
Repository	circuit-breaker
Commit	a9e4d1e30faf...
Fix Commit	30b01f13792e...
Audit Timeline	Apr 6th - Apr 7th, 2026
Methods	Manual Review, Formal Verification

Issues Found

Critical Risk	0
High Risk	0
Medium Risk	0
Low Risk	0
Informational	2
Gas Optimizations	3
Total Issues	5

Summary of Findings

[I-1] Missing named mapping value parameters in `Registry.Storage`	Resolved
[I-2] Single pauser per pausable combined with heartbeat expiry lockout leaves zero emergency coverage during liveness gaps	Acknowledged
[G-1] Emit before state change to eliminate temporary variable	Resolved
[G-2] Solidity optimizer is not enabled	Resolved
[G-3] Redundant storage read of `registry.pauser[_pausable]` in `CircuitBreaker::pause`	Acknowledged

7 Findings

7.1 Informational

7.1.1 Missing named mapping value parameters in `Registry.Storage`

Description: Three mappings in `Registry.Storage` declare named keys but omit names for their value types, reducing readability and ABI clarity:

```
// Registry.sol lines 18-21
mapping(address pausable => address) pauser;
mapping(address pausable => uint256) oneBasedIndex;
mapping(address pauser => uint256) pausableCount;
```

Recommended Mitigation:

```
-mapping(address pausable => address) pauser;
-mapping(address pausable => uint256) oneBasedIndex;
-mapping(address pauser => uint256) pausableCount;
+mapping(address pausable => address pauser) pauser;
+mapping(address pausable => uint256 oneBasedIndex) oneBasedIndex;
+mapping(address pauser => uint256 pausableCount) pausableCount;
```

Lido: Fixed in commit cabcfec.

Cyfrin: Verified.

7.1.2 Single pauser per pausable combined with heartbeat expiry lockout leaves zero emergency coverage during liveness gaps

Description: Each pausable contract can only have one configured pauser in `CircuitBreaker`. If that pauser's heartbeat expires, they are completely locked out — both `CircuitBreaker::heartbeat` and `CircuitBreaker::pause` require `isPauserLive` to return true (via `_updateHeartbeat` with `_requireActive = true`):

```
// CircuitBreaker.sol
    function heartbeat() external {
        require(registry.isRegistered(msg.sender), SenderNotPauser());
        _updateHeartbeat(msg.sender, true);  // requires live
    }

    function pause(address _pausable) external nonReentrant {
        require(msg.sender == registry.getPauser(_pausable), SenderNotPauser());
        _updateHeartbeat(msg.sender, true);  // requires live

    function _updateHeartbeat(address _pauser, bool _requireActive) internal {
        if (_requireActive) require(isPauserLive(_pauser), HeartbeatExpired());
```

The only way to restore an expired pauser is the admin calling `CircuitBreaker::registerPauser` to re-register them — this calls `_updateHeartbeat(_newPauser, false)` which bypasses the liveness check.

This creates a gap: between heartbeat expiry and admin re-registration, the pausable contract has zero emergency pause coverage through `CircuitBreaker`. If an emergency occurs during this window, the slow governance path is the only option — exactly the scenario `CircuitBreaker` was built to prevent.

The design rationale is sound ("A committee that cannot prove its liveness should not be trusted to respond in an emergency"), but the single-pauser-per-pausable constraint means there is no fallback. Consider whether the `Registry` contract used by `CircuitBreaker` should support multiple pausers for each pausable contract.

Lido: Acknowledged as:

- Single pauser keeps accountability clear

- Expiry is the enforcement mechanism; a committee that can't send one heartbeat transaction once per year (for example) shouldn't hold pause authority
- Heartbeat expiry is fully trackable; off-chain monitoring can alert pausers months in advance before expiry
- the tradeoff of having no fallback is accepted

7.2 Gas Optimization

7.2.1 Emit before state change to eliminate temporary variable

Description: `CircuitBreaker::_setPauseDuration` and `CircuitBreaker::_setHeartbeatInterval` each declare a `previous*` local variable solely to capture the old storage value for the event. Emitting the event before the state change lets the event read the current storage value directly, eliminating the temporary:

```
// CircuitBreaker.sol lines 289-292
uint256 previousPauseDuration = pauseDuration;
pauseDuration = _newPauseDuration;
emit PauseDurationUpdated(previousPauseDuration, _newPauseDuration);

// CircuitBreaker.sol lines 300-303
uint256 previousHeartbeatInterval = heartbeatInterval;
heartbeatInterval = _newHeartbeatInterval;
emit HeartbeatIntervalUpdated(previousHeartbeatInterval, _newHeartbeatInterval);
```

Recommended Mitigation:

```
 function _setPauseDuration(uint256 _newPauseDuration) internal {
     require(_newPauseDuration >= MIN_PAUSE_DURATION, PauseDurationBelowMin());
     require(_newPauseDuration <= MAX_PAUSE_DURATION, PauseDurationAboveMax());
-    uint256 previousPauseDuration = pauseDuration;
+    emit PauseDurationUpdated(pauseDuration, _newPauseDuration);
     pauseDuration = _newPauseDuration;
-    emit PauseDurationUpdated(previousPauseDuration, _newPauseDuration);
 }
```

Same pattern for `CircuitBreaker::_setHeartbeatInterval`.

Lido: Fixed in commit 11aea85.

Cyfrin: Verified.

7.2.2 Solidity optimizer is not enabled

Description: The Foundry configuration does not enable the Solidity optimizer:

```
foundry.toml
[profile.default]
src = "src"
out = "out"
libs = ["lib"]
solc = "0.8.34"
```

No `optimizer` or `optimizer_runs` keys are present. Foundry disables the optimizer by default when these are absent. Even at the default 200 runs, the optimizer eliminates dead code, simplifies constant expressions, and reduces both deployment and runtime gas.

Recommended Mitigation: Add optimizer settings to `foundry.toml`:

```
 [profile.default]
 src = "src"
 out = "out"
 libs = ["lib"]
 solc = "0.8.34"
+optimizer = true
+optimizer_runs = 10000
```

`optimizer_runs` controls the trade-off between deployment cost and runtime call cost. Lower values (e.g., 1) produce smaller bytecode that is cheaper to deploy but slightly more expensive per call. Higher values (e.g., 10,000+) produce larger bytecode optimized for cheaper calls at the cost of more expensive deployment.

Factory-deployed contracts benefit from lower values because deployment cost is paid every time the factory creates a new instance — with hundreds of deployments the cumulative savings outweigh the marginal per-call increase.

Core protocol contracts deployed once but called millions of times benefit from higher values since deployment is a one-time expense. `CircuitBreaker` falls into the latter category — it is deployed once and called repeatedly — so a higher `optimizer_runs` value (e.g., 10,000) is appropriate. The default 200 is a reasonable starting point if unsure.

Lido: Fixed in commit 30b01f1.

Cyfrin: Verified.

7.2.3 Redundant storage read of `registry.pauser[_pausable]` in `CircuitBreaker::pause`

Description: In `CircuitBreaker::pause`, the storage slot `registry.pauser[_pausable]` is read twice via separate library calls:

```
CircuitBreaker.sol
    function pause(address _pausable) external nonReentrant {
        require(msg.sender == registry.getPauser(_pausable), SenderNotPauser());  // SLOAD 1
        _updateHeartbeat(msg.sender, true);

        uint256 duration = pauseDuration;
        IPausable target = IPausable(_pausable);

        registry.setPauser(_pausable, address(0));  // SLOAD 2 (re-reads pauser[_pausable] at
↪  Registry.sol:85)
```

`Registry::getPauser` reads `_self.pauser[_pausable]` (Registry.sol:44) for the authorization check. Then `Registry::setPauser` reads the same slot again (Registry.sol:85: `address previousPauser = _self.pauser[_pausable]`) to determine the previous pauser for count management. The value has not changed between reads, so the second SLOAD (100 gas warm) is redundant.

Recommended Mitigation: `Registry::setPauser` already reads `previousPauser` — return it via a named return value and use it in `CircuitBreaker::pause` to combine the authorization check with the unregistration:

```
 // Registry.sol
-function setPauser(Storage storage _self, address _pausable, address _newPauser) internal {
+function setPauser(Storage storage _self, address _pausable, address _newPauser) internal returns
↪  (address previousPauser) {
     require(_pausable != address(0), PausableZero());

-    address previousPauser = _self.pauser[_pausable];
+    previousPauser = _self.pauser[_pausable];

     // ... existing logic unchanged ...
 }
```

```
 // CircuitBreaker.sol
 function pause(address _pausable) external nonReentrant {
-    require(msg.sender == registry.getPauser(_pausable), SenderNotPauser());
+    require(msg.sender == registry.setPauser(_pausable, address(0)), SenderNotPauser());
     _updateHeartbeat(msg.sender, true);

     uint256 duration = pauseDuration;
     IPausable target = IPausable(_pausable);

-    registry.setPauser(_pausable, address(0));
     target.pauseFor(duration);
     require(target.isPaused(), PauseFailed());
```

```
    emit PauseTriggered(_pausable, msg.sender, duration);
}
```

Lido: Acknowledged; 100 gas saved is negligible compared to the decreased readability for a function call that we hope will never be called. The proposed fix:

- hides state mutation inside an auth check
- returns the previous pauser from `setPauser`, which breaks the standard convention where a setter usually returns the new value (or a success boolean)